

datalab 报告

姓名：梁恒溢

学号：2025201739

总分 bitXor logtwo byteSwap reverse ...

37 1 4 4 3 . . .

test 截图：

```
lianghengyi@lianghengyi:~/datalab2026_lhy-1$ make
python3 test.py
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
rm -f *.o btest fshow ishow *~ yacstab.py lextab.py result.txt .autograder_result
gcc -O0 -Wall -std=gnu99 -lm -o btest bits.c btest.c decl.c tests.c
gcc -O0 -Wall -std=gnu99 -o fshow fshow.c
gcc -O0 -Wall -std=gnu99 -o ishow ishow.c
Make success.
bitAnd          1/1:      PASS
bitXor          1/1:      PASS
samesign        2/2:      PASS
logtwo          4/4:      PASS
byteSwap        4/4:      PASS
reverse         3/3:      PASS
logicalShift    3/3:      PASS
leftBitCount    4/4:      PASS
float_i2f       4/4:      PASS
floatScale2     4/4:      PASS
float64_f2i     3/3:      PASS
floatPower2     4/4:      PASS
Total points: 37
```

解题报告

亮点

1. bitXor
2. logtwo
3. logicalshift

bitXor

```
// int bitXor(int x, int y) {
    return (~x&~y)&^(x&y);
}
```

这道题并不复杂，就是“非二者同为0且非二者同为1”，我只是从中感受到了~按位取反和&按位与可以不假思索地去当作！和&&使用，只是对位操作而已，某种角度上像不假思索的写递归一样，直接就实现了正确答案

logtwo

```
int logtwo(int v) {
    int shift=0;
    int result=0;

    shift=(v>0xFFFF)<<4;
    result=result|shift;
    v=v>>shift;
```

```
    shift=(v>0xFF)<<3;
    result=result|shift;
    v=v>>shift;
    shift=(v>0xF)<<2;
    result=result|shift;
    v=v>>shift;
    shift=(v>0x3)<<1;
    result=result|shift;
    v=v>>shift;
    shift=(v>0x1);
    result=result|shift;

    return result;
}
```

这道题一方面和后面的leftbitcount一样，因为不能用循环就用了二分的方法，去找最高的1在第几位，一方面。因为所有数都能用2的各个次幂选取并求和来表示，保证了算法正确性；另一方面，因为shift只能由=有一位是1，所以反而用result|shift这一或运算实现了加法；而且shift是1自然左移得到对应2的次幂，为0左移后仍为0也很漂亮

logicalshift

```
int logicalShift(int x, int n) {
    return x>>n&^(((1<<31)>>n)<<1);
    //return x>>n&^(~0<<(32-n));
    //return x>>n&^((1<<(32-n))+~0);
}
```

这道题关键就是算术右移后前面会补符号位，因此要把左起n位和全0与一次，关键就成了构造左起n位全0其他位全1的字串，把1移至最左再右移对应位可天然地再左边全部补上1，非常精妙，是正确结果。我注释的第二三个方法也可，但会在n==0时失效，因此有样例没通过。

反馈/收获/感悟/总结

大概用了一个下午，我感觉一方面是细节问题很多，包括比如32-n还是31-n，从0还是从1开始之类的小问题，都得举个例子直观推比较可靠；另一方面就是对位运算还不太熟悉，虽然课上讲的都可以理解接受，但在实际实现中，尤其是循环和if不能用的情况下，很多功能实现都缺乏思路，比如二分的想法就是AI给的。但也体会到了位运算的一些妙处，比如logicalshift里1移到最左后再右移n位就直接构造出了左起n+1位都为1其他位为0的字串等。不合理的地方的话我感觉对于究竟需要提交什么，提交的东西里面要包含什么说的还不是太清晰。

参考的重要资料

基本都是照着PPT的知识以及AI辅助思路，没什么参考资料